

Architectural Blueprint and Engineering Guidelines for a Distributed ESP32 Smart Home Ecosystem

The deployment of a fully automated, hardware-optimized smart home system requires a robust architecture capable of balancing ultra-low power consumption at the edge with high-reliability data aggregation at the core. Standard 802.11 Wi-Fi networks are fundamentally unsuited for battery-powered, highly responsive sensor networks due to the immense energetic overhead required for DHCP handshakes, router association, and TCP/IP stack maintenance. To circumvent these limitations, the following engineering guideline provides an exhaustive, end-to-end blueprint for constructing a distributed smart home ecosystem. This architecture leverages the ESP32-C3 SuperMini for low-power sensor nodes, the ESP-NOW protocol for connectionless RF communication, an always-on ESP32 gateway bridging RF to Home Assistant via a physical serial connection, and high-voltage solid-state relays powered by isolated AC-DC converters for actuator nodes. This document serves as a complete methodology, covering component selection, deep-sleep firmware engineering, high-voltage safety paradigms, and localized supply chain logistics.

1. System Architecture Topography and Physical Layer Constraints

The ecosystem is divided into distinct operational layers, ensuring that computationally heavy tasks are decoupled from power-constrained edge devices. At the furthest edge of the network are ultra-low-power sensor nodes, which are battery and solar-powered devices utilizing the ESP32-C3 SuperMini. These nodes remain in a deep sleep coma, waking only via hardware interrupts triggered by passive infrared (PIR) motion detection to broadcast a transient data payload before immediately returning to sleep. The transport layer relies entirely on the connectionless RF protocol known as ESP-NOW. This is a low-latency, peer-to-peer Wi-Fi protocol that eliminates the prolonged handshakes of standard Wi-Fi, allowing edge nodes to transmit data in under ten milliseconds.

Because a central server cannot natively decode ESP-NOW 802.11 action frames, a translation gateway node is required. This is an always-on, mains-powered ESP32 device that promiscuously listens for ESP-NOW broadcasts, formats the raw byte payloads into delimited strings or JSON, and passes them to the central server via a physical Universal Asynchronous Receiver-Transmitter (UART) connection. The central automation brain is a localized server, specifically a Raspberry Pi 5 operating Home Assistant. This server ingests the serialized data, parses it via YAML templating, updates the internal state machine, and triggers environmental automations. Finally, the system relies on high-voltage actuator nodes. These are ESP32 devices directly interfaced with mains power (220V AC) using Hi-Link HLK-PM01 AC-DC converters and Omron G3MB-202P Solid State Relays (SSRs) to safely drive inductive loads such as ceiling fans and heavy lighting without the aesthetic penalty of external power

adapters.

The physical environment poses a significant challenge to radio frequency propagation. In typical South Asian architectural environments, such as those found in Bangladesh, internal walls are constructed from dense brick and reinforced concrete. A 2.4GHz signal emitted by a trace antenna on a microscopic PCB will experience severe attenuation when penetrating multiple concrete barriers. If the Raspberry Pi 5 Home Assistant server is located in a peripheral room, such as a reading room, a low-power transmission from a distant balcony sensor will likely fail to reach the gateway. To overcome this physics limitation without resorting to power-hungry mesh networking algorithms, the architecture utilizes a strategic relay topology. The smart light switches deployed in intermediate rooms operate on 220V AC mains power and are therefore awake continuously. These actuator nodes are programmed with dual functionality: they listen for their own automation commands while simultaneously acting as transparent repeaters for the battery-powered edge nodes, catching weak signals from the balcony and blasting them forward to the central gateway.

2. Hardware Engineering: Ultra-Low Power Sensor Nodes

The foundation of the sensor network relies on minimizing the quiescent current of the hardware. The ESP32-C3 SuperMini is selected for its 32-bit RISC-V single-core processor running at 160 MHz, offering a highly compact form factor and advanced deep sleep capabilities.¹

2.1 Sensor Node Component Specifications

To construct a sensor node that can operate autonomously for years without intervention, every component must be scrutinized for parasitic power draw. The table below outlines the primary components required for the edge nodes, focusing on their energetic profiles.

Component Designation	Functional Role	Architectural Justification and Energetic Profile
ESP32-C3 SuperMini	Microcontroller	Utilizes the RISC-V architecture operating natively on 3.3V logic. Contains 400KB SRAM and an integrated Wi-Fi/Bluetooth subsystem. Theoretical deep sleep current is approximately 43 μ A.

AM312 PIR Module	Motion Detection	Draws a maximum of 15 μ A to 20 μ A quiescent current, vastly outperforming the ubiquitous HC-SR501 which draws over 50 μ A. ³ Operates natively at 3.3V, eliminating logic level shifters. ⁵
18650 Li-Ion Cell	Energy Storage	High volumetric energy density providing 2500mAh to 3500mAh nominal capacity. Capable of powering a deeply optimized edge node for 18 to 24 months per charge cycle. ³
TP4056 Module	Battery Management	Provides a constant-current/constant-voltage charging profile up to 1A with integrated over-discharge protection terminating at 2.5V to prevent cell degradation. ⁷
MCP1700-3302E	LDO Voltage Regulator	Steps down the variable 3.7V-4.2V lithium-ion battery voltage to a stable 3.3V for the microcontroller. Features an exceptionally low quiescent current compared to standard AMS1117 regulators. ⁸

2.2 Deep Sleep Optimization and Parasitic Drain Eradication

Out of the box, standard ESP32-C3 SuperMini development boards are not optimized for micro-ampere deep sleep due to the inclusion of parasitic board-level components intended for debugging and status indication. An unmodified ESP32-C3 SuperMini typically draws

between 671 μA and 1.5 mA during deep sleep, a figure that will drain an 18650 battery in a matter of months.¹⁰ To achieve a true ultra-low power state, aggressive hardware modification is mandatory.

The primary parasitic draw is the always-on red surface-mount device (SMD) power indicator LED and the WS2812 addressable RGB LED.² The red power LED is tied directly to the voltage rail through a current-limiting resistor, usually 5.1k Ω , and cannot be disabled programmatically through the ESP-IDF or Arduino frameworks.¹⁰ Removing this LED and its associated resistor via desoldering or physical crushing eliminates approximately 300 μA to 450 μA of continuous current draw.¹⁰ Furthermore, while the WS2812 RGB LED is tied to GPIO8 and can have its data line pulled low, the internal controller of the WS2812 chip continuously consumes power.¹³ Physically removing the RGB LED from the printed circuit board further drops the deep sleep current to an astonishing 9 μA to 43 μA , allowing the node to reach theoretical maximum efficiency.²

2.3 The Sustainable Power Subsystem: Solar and Load Sharing

For continuous autonomous operation, particularly on balconies where direct sunlight is available, an 18650 lithium-ion battery must be coupled with a miniature 5V or 6V solar panel.¹⁴ The solar panel is wired into the input terminals of the TP4056 charger module, which governs the charging phases of the battery connected to its battery terminals.⁸

However, drawing continuous load from the output terminals of the TP4056 while it simultaneously attempts to charge the battery from the solar panel creates a critical failure in the charge termination algorithm.¹⁶ The TP4056 determines that a battery is fully charged when the charging current drops below a specific threshold, typically 1/10th of the programmed charging current. If the ESP32 load continuously draws current, the TP4056 will never register a drop below this threshold, causing it to continuously pump a float charge into the lithium-ion cell, ultimately degrading its chemistry and creating a severe fire hazard.¹⁸

To safely bypass this limitation, the engineering standard requires a load-sharing circuit, often referred to as a power path management circuit. This is constructed using a logic-level P-Channel MOSFET (such as an NDP6020P or IRL3103PBF), a Schottky diode (such as a 1N5819), and a pull-down resistor.⁹ In this configuration, the solar panel is connected to the gate of the P-Channel MOSFET. When the solar panel is producing voltage, it applies a high logic level to the gate, forcing the MOSFET into a non-conducting state and blocking the battery from discharging.²⁰ The solar panel simultaneously powers the load directly through the Schottky diode, which prevents reverse current flow during darkness, and charges the battery via the TP4056.²⁰ When the solar voltage drops below the battery voltage, the MOSFET gate is pulled low by the resistor, turning the transistor on and allowing the battery to seamlessly take over powering the ESP32 without resetting the microcontroller.²⁰

2.4 PIR Sensor Integration and Trigger Mechanics

The AM312 passive infrared sensor operates on a 3.3V supply and outputs a 3.3V logic high signal when motion is detected.²¹ Because the ESP32-C3 operates strictly on 3.3V logic, the

AM312 output pin can be wired directly to the microcontroller without the need for logic level shifters, which would otherwise introduce additional quiescent current.⁵

The output of the AM312 must be connected to an RTC-capable GPIO pin. On the ESP32-C3, the RTC domain is restricted to GPIO0 through GPIO5.²³ When the system enters deep sleep, the main CPU, wireless peripherals, and primary RAM are powered down, leaving only the ultra-low-power RTC controller active to monitor these specific pins.²⁴ To prevent false triggers caused by electromagnetic interference or voltage droop during the high-current Wi-Fi transmission bursts, hardware debouncing is highly recommended. Placing a 100nF ceramic capacitor and a 10k Ω pull-down resistor on the GPIO input line creates a low-pass filter that absorbs transient noise, ensuring that only genuine, sustained infrared signatures wake the device.

3. Firmware Engineering: The Edge Node Software Architecture

The firmware for the edge nodes is dictated by the absolute necessity to minimize awake time. ESP-NOW is a connectionless protocol that encapsulates application data in a vendor-specific 802.11 action frame.²⁵ Because it does not require a router association or an IP address lease, transmission occurs almost instantly.⁶ The maximum packet length for ESP-NOW is 250 bytes, which is vastly more than required to transmit a sensor state, a device identifier, and a battery voltage metric.²⁵

3.1 The RISC-V Deep Sleep Paradigm

Programming deep sleep interrupts on the ESP32-C3 (RISC-V architecture) deviates significantly from the legacy ESP32 (Xtensa architecture). While older chips utilized the `esp_sleep_enable_ext0_wakeup` command, the C3 architecture requires the `esp_deep_sleep_enable_gpio_wakeup` function.²⁷ Furthermore, the C3 must have the pin explicitly configured for RTC monitoring prior to entering sleep.

The architectural flow of the firmware ensures that the `loop()` function is entirely empty. When the PIR sensor pulls the RTC GPIO pin high, the ESP32-C3 boots, executes the `setup()` block, transmits the data, configures the wake-up parameters, and immediately calls `esp_deep_sleep_start()`.²⁷ This methodology ensures the microcontroller remains in an active, high-power state for fewer than 150 milliseconds.

3.2 C++ Implementation for the Sensor Node

The following source code represents the exhaustive implementation for the battery-powered edge node. It is designed to be compiled within the Arduino IDE or PlatformIO environment.

```

C++
#include <esp_now.h>
#include <WiFi.h>
#include <driver/rtc_io.h>

// The MAC address of the destination (Repeater or Gateway)
uint8_t targetAddress = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};

// Data structure matching the receiver's expected format
typedef struct payload_structure {
    uint8_t device_id;
    bool motion_detected;
    float battery_voltage;
} payload_structure;

payload_structure sensorData;
esp_now_peer_info_t peerInfo;

// Define the RTC capable wake-up pin
#define PIR_PIN GPIO_NUM_3
#define BATTERY_PIN GPIO_NUM_4

void setup() {
    // Read the cause of the wake-up before any initialization changes state
    esp_sleep_wakeup_cause_t wakeup_reason = esp_sleep_get_wakeup_cause();

    // Initialize Wi-Fi in Station mode but strictly disconnect from any AP
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    // Initialize the ESP-NOW protocol
    if (esp_now_init() != ESP_OK) {
        // If initialization fails, fall back to sleep immediately to save power
        goToSleep();
    }

    // Register the peer device
    memcpy(peerInfo.peer_addr, targetAddress, 6);
    peerInfo.channel = 1; // Channel must match the gateway/repeater precisely
    peerInfo.encrypt = false; // Disable encryption to minimize computational overhead

    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
        goToSleep();
    }
}

```

```

// Assign static device ID (e.g., 1 for Balcony)
sensorData.device_id = 1;

// Determine if wake-up was caused by the PIR sensor or a timer
sensorData.motion_detected = (wakeup_reason == ESP_SLEEP_WAKEUP_GPIO)? true
false;

// Perform a rapid ADC read to ascertain battery voltage
// Utilizing a voltage divider circuit to drop 4.2V below the 3.3V ADC limit
int raw_adc = analogRead(BATTERY_PIN);
sensorData.battery_voltage = (raw_adc / 4095.0) * 3.3 * 2 // Assuming a 1:1 voltage divider

// Transmit the action frame via ESP-NOW
esp_err_t result = esp_now_send(targetAddress, (uint8_t *) &sensorData,
sizeof(sensorData));

// A brief delay ensures the RF hardware completes transmission before power is cut
delay(10);

// Terminate execution and power down
goToSleep();
}

void loop() {
// Empty by design to enforce rapid sleep cycling
}

void goToSleep() {
// Explicitly power down the RF subsystem
esp_now_deinit();
WiFi.mode(WIFI_OFF);

// Configure the RTC GPIO for the RISC-V architecture
pinMode(PIR_PIN, INPUT);
esp_deep_sleep_enable_gpio_wakeup(1ULL << PIR_PIN, ESP_GPIO_WAKEUP_GPIO_HIGH);

// Optional: Set a timer wake-up as a heartbeat to report battery health
// esp_sleep_enable_timer_wakeup(3600 * 1000000ULL); // Wake every 1 hour

// Enter the deep sleep coma
esp_deep_sleep_start();
}

```

The design ensures that if the destination device does not exist or the channel is mismatched, the system will not hang in an infinite loop waiting for an acknowledgment, but will safely abort and return to sleep.³⁰

4. Hardware Engineering: High-Voltage Actuator Nodes

While edge sensors rely on extreme low-power metrics to survive on chemical batteries, actuator nodes, such as smart switches designed to control lighting, ceiling fans, or dehydrators, are intrinsically tied to the domestic mains voltage network (220V AC). In these environments, power efficiency is secondary to severe requirements for galvanic isolation, thermal management, and surge protection.

4.1 Converting AC Mains to DC Logic Power (HLK-PM01)

To eliminate the aesthetic and spatial penalties of external USB power adapters, the actuator nodes utilize PCB-mounted step-down power supply modules. The Hi-Link HLK-PM01 is an ultra-compact, encapsulated module that accepts a wide alternating current input (90~264V AC) and provides a highly stable 5V direct current output capable of delivering up to 3 Watts or 600mA.³¹ This is more than sufficient to power an ESP32 and trigger an array of solid-state relays.

However, integrating the HLK-PM01 directly into a custom circuit board inside a domestic wall box introduces profound life-safety risks. Encapsulated switch-mode power supplies can fail catastrophically, often resulting in internal short circuits that draw massive currents from the mains, leading to spontaneous combustion.³³ The manufacturer datasheet strictly dictates the inclusion of peripheral protection circuitry, which must never be omitted in DIY designs.³¹

The mandated protection circuit includes a 0.5A or 1A slow-blow fuse placed in series on the AC Live line to break the circuit permanently in the event of an internal short.³¹ To absorb high-voltage transients and lightning strikes, a metal-oxide varistor (MOV), specifically the 10D561K, must be placed in parallel across the AC Live and Neutral lines.³⁴ When a voltage spike exceeds the varistor's clamping voltage, its resistance drops exponentially, short-circuiting the transient and blowing the fuse, thereby protecting the downstream electronics. Furthermore, a 10mH to 15mH common mode choke is required to prevent high-frequency switching noise generated by the HLK-PM01 from bleeding back into the domestic mains supply, which could otherwise interfere with adjacent electronics.³⁴

When designing the custom printed circuit board for the actuator node, the engineer must maintain strict physical separation between the high-voltage AC tracks and the low-voltage 5V/3.3V DC tracks. This involves routing the traces to maintain a minimum creepage distance of 5mm to 8mm and physically milling slots into the fiberglass PCB material beneath the HLK-PM01 to prevent high-voltage arcing across the board surface.³⁵

4.2 Solid-State AC Switching (Omron G3MB-202P)

To switch the 220V alternating current flowing to the home appliances, the architecture discards traditional electromechanical relays (EMRs), which are bulky, prone to contact arcing, and emit a loud clicking noise. Instead, it utilizes Solid State Relays (SSRs), specifically the Omron G3MB-202P, which provides silent, zero-cross switching of AC loads up to 2A at 240V AC.³⁶

The solid-state relay achieves galvanic isolation through an internal phototriac coupler.³⁶ When the ESP32 supplies a logic signal to the SSR, it illuminates an internal light-emitting diode. The emitted photons travel across an insulating physical gap within the chip casing and strike a photosensitive semiconductor, which then permits the high-voltage AC current to flow across the load terminals.³⁶ This optical isolation guarantees that no catastrophic fault on the 220V side can back-feed electrical current into the delicate 3.3V ESP32 circuitry.

The Omron G3MB-202P incorporates zero-cross functionality, meaning it waits for the precise millisecond the sinusoidal AC voltage wave crosses zero volts before completing the circuit.³⁶

This drastically reduces inrush currents and eliminates electrical noise. However, driving inductive loads, such as the electric motors found in ceiling fans, presents a unique hazard.

When an inductive load is disconnected, the collapsing magnetic field within the motor coils generates a massive voltage spike known as back electromotive force (back EMF).³⁹ If this spike exceeds the breakdown voltage of the triac inside the SSR, the relay will be permanently destroyed, often failing in a closed state, which leaves the appliance permanently turned on.³⁹

While the G3MB-202P contains an internal snubber circuit designed to suppress rapid voltage changes ($dv/dt = 100 \text{ V}/\mu\text{s}$)³⁶, robust engineering dictates the addition of a supplementary external RC snubber circuit across the SSR's load terminals for heavy motors. This consists of a 100Ω 1W resistor in series with a 0.1μF 400V capacitor, which safely absorbs and dissipates the inductive kickback.⁴⁰

Because the internal LED of the G3MB-202P requires 5V to fully illuminate and ensure strong switching action, relying on the ESP32's 3.3V GPIO pins can lead to unreliable triggering where the relay hovers in a partially conductive state.⁴¹ Therefore, a low-side switching topology must be employed.⁴² An NPN transistor, such as a BC547, is placed between the SSR's negative input terminal and ground.⁴⁰ The ESP32's 3.3V signal drives the base of the transistor through a 10kΩ current-limiting resistor, saturating the transistor and pulling the SSR's terminal to ground, thereby completing the 5V circuit provided by the HLK-PM01.⁴³

5. Firmware Engineering: The Mesh Repeater Topology

Because the ESP-NOW protocol operates on the 2.4GHz spectrum, signals from distant balconies will fail to penetrate multiple concrete walls to reach the central gateway. To solve this, the high-voltage actuator nodes located in intermediate rooms are tasked with acting as signal repeaters. Because these nodes are powered by the HLK-PM01 from the mains supply, power conservation is irrelevant, and they can remain actively listening for RF packets indefinitely.

The repeater firmware is designed to intercept packets destined for the central gateway, inspect the payload, and re-transmit the exact same action frame to bridge the physical gap.

5.1 C++ Implementation for the Repeater Node

The code below outlines the logic for an intermediate smart switch. It handles its own automation responsibilities (switching the local solid-state relay) while implementing the OnDataRecv callback to immediately execute an esp_now_send command, effectively forwarding the telemetry of the battery-powered edge nodes.

```
C++
#include <esp_now.h>
#include <WiFi.h>

// MAC Address of the final destination (The Central Gateway)
uint8_t gatewayAddress = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};

// MAC Address of the edge sensor this repeater is listening for
// In a full implementation, an array of authorized MACs should be validated
uint8_t edgeSensorMAC = {0x11, 0x22, 0x33, 0x44, 0x55, 0x66};

esp_now_peer_info_t peerInfo;

// Define the pin connected to the NPN transistor driving the Omron SSR
#define RELAY_PIN GPIO_NUM_22

// The structure must identically match all nodes in the ecosystem
typedef struct payload_structure {
    uint8_t device_id;
    bool motion_detected;
    float battery_voltage;
} payload_structure;

payload_structure incomingData;

// Callback triggered upon receiving an ESP-NOW packet
void OnDataRecv(const uint8_t * mac_addr, const uint8_t *incomingDataPtr, int len)
{
    // Validate packet length to prevent buffer overflow attacks
    if (len != sizeof(payload_structure)) {
        return
    }
}
```

```

// Determine if the packet originated from the authorized edge sensor
bool isAuthorized = true
for (int i = 0; i < 6; i++) {
    if (mac_addr[i] != edgeSensorMAC[i]) {
        isAuthorized = false
        break
    }
}

if (isAuthorized) {
    // Copy the memory block into the local structure
    memcpy(&incomingData, incomingDataPtr, sizeof(incomingData))

    // The repeater acts as a transparent bridge, instantly forwarding the packet
    esp_err_t result = esp_now_send(gatewayAddress, (uint8_t *) &incomingData,
    sizeof(incomingData));

    if (result == ESP_OK) {
        Serial.println("Packet successfully relayed to central gateway.");
    }
}

// Callback to handle status of relayed messages
void onDataSent(const uint8_t *mac_addr, esp_now_send_status_t status) {
    Serial.print("Relay Transmission Status: ");
    Serial.println(status == ESP_NOW_SEND_SUCCESS? "Success" : "Fail");
}

void setup() {
    Serial.begin(115200);
    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, LOW); // Ensure relay defaults to OFF

    // Initialize Wi-Fi in Station mode.
    // It is critical to enforce the same Wi-Fi channel across all nodes.
    WiFi.mode(WIFI_STA);

    if (esp_now_init() != ESP_OK) {
        Serial.println("Error initializing ESP-NOW");
        return
    }
}

```

```

// Register callbacks
esp_now_register_rcv_cb(OnDataRecv);
esp_now_register_send_cb(OnDataSent);

// Register the central gateway as a peer to allow forwarding
memcpy(peerInfo.peer_addr, gatewayAddress, 6);
peerInfo.channel = 1;
peerInfo.encrypt = false;

if (esp_now_add_peer(&peerInfo) != ESP_OK) {
    Serial.println("Failed to add gateway peer");
    return;
}

void loop() {
    // The repeater remains perpetually awake.
    // Code to toggle the RELAY_PIN based on local physical switches or
    // direct commands from the central gateway would execute here.
}

```

By ensuring that the repeater does not modify the payload structure, the central gateway remains agnostic to the routing path; it simply processes the incoming variables as if the edge node had communicated directly.⁴⁴

6. The Central Translation Gateway Node

The terminal point of the RF network is the translation gateway, an ESP32 plugged directly into the Raspberry Pi 5 via a Universal Serial Bus (USB) cable. The Home Assistant software running on the server is blind to the 802.11 action frames used by ESP-NOW. It relies on standard protocols like MQTT or direct serial ingestion.⁴⁶

While it is possible to build a gateway that connects to the local Wi-Fi router to publish MQTT messages⁴⁸, doing so forces the ESP32 to maintain a connection to the router while simultaneously receiving ESP-NOW packets. This creates a severe conflict: modern domestic routers implement channel hopping, constantly shifting their 2.4GHz frequency to avoid interference. Because ESP-NOW requires all nodes to communicate on a strictly fixed channel, the moment the router changes channels, the ESP32 gateway loses its connection to either the router or the sensor nodes.⁴⁹ Resolving this via dual-chip MQTT gateways is overly complex. The elegant, robust solution is the Serial Bridge.⁵¹ By pushing the data across the USB cable directly into the Raspberry Pi, the entire smart home infrastructure operates completely independent of the local area network, immunizing the system against Wi-Fi outages and router instability.

6.1 Gateway Antenna Configuration

Because the gateway resides centrally, maximizing its radio sensitivity is beneficial. Using a standard ESP32 with a PCB trace antenna provides limited gain. The optimal hardware choice is an ESP32-WROOM-32U module, which is equipped with a miniature U.FL (IPEX) connector.⁵³ This allows the attachment of a physical, external 2.4GHz omnidirectional antenna, significantly increasing the node's signal-to-noise ratio and reception range. To utilize the U.FL connector, the manufacturer sometimes requires an onboard zero-ohm resistor to be physically desoldered and shifted to bridge the trace leading to the connector rather than the internal antenna, though many modern boards are pre-configured.⁵³

6.2 C++ Implementation for the Serial Gateway

The gateway firmware is lightweight. Its sole purpose is to promiscuously accept validated ESP-NOW packets, serialize the variables into a comma-separated format, and push them to the hardware UART buffer.

```
C++
#include <esp_now.h>
#include <WiFi.h>

// Must match the payload structure exactly
typedef struct payload_structure {
    uint8_t device_id;
    bool motion_detected;
    float battery_voltage;
} payload_structure;

payload_structure incomingData;

// The callback executes asynchronously upon packet receipt
void OnDataRecv(const uint8_t * mac_addr, const uint8_t *incomingDataPtr, int len)
{
    if (len != sizeof(payload_structure)) {
        return;
    }

    memcpy(&incomingData, incomingDataPtr, sizeof(incomingData));

    // Serialize the data into a comma-separated string over the USB interface
```

```

// Format: "HEADER,DEVICE_ID,MOTION_STATE,VOLTAGE"
Serial.print("ESPNOW_RCV,");
Serial.print(incomingData.device_id);
Serial.print(",");
Serial.print(incomingData.motion_detected);
Serial.print(",");
Serial.println(incomingData.battery_voltage);
}

void setup() {
// Match the baud rate to Home Assistant's configuration
Serial.begin(115200);

WiFi.mode(WIFI_STA);
// Lock the gateway to the agreed-upon channel to prevent drift
esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);

if (esp_now_init() != ESP_OK) {
Serial.println("Gateway ESP-NOW Init Failure");
return
}

esp_now_register_recv_cb(OnDataRecv);

}

void loop() {
// The gateway yields to the RTOS, handling callbacks in the background
delay(10);
}

```

7. Core Integration: Home Assistant Configuration and Templating

Once the serial gateway is physically plugged into the Raspberry Pi 5, the Linux operating system will enumerate the device, typically assigning it to `/dev/ttyUSB0` or `/dev/ttyACMO`.⁵⁵ The Home Assistant core must be configured to open a serial stream to this device, intercept the incoming string, and disaggregate the variables into independent entities representing the physical state of the house.

7.1 Splitting the Configuration Architecture

Constructing the data parser within a single `configuration.yaml` file results in an unmanageable, monolithic block of code that severely hampers debugging.⁵⁶ The professional standard is to

modularize the architecture by splitting the configuration utilizing YAML !include directives.⁵⁶ The primary configuration.yaml is modified to point to subsidiary files:

```
YAML
# configuration.yaml
default_config:

# Initialize the serial port listener
sensor:!include sensors.yaml

# Delegate template logic to a dedicated file
template:!include templates.yaml
```

7.2 Defining the Base Serial Sensor

Within the sensors.yaml file, the serial integration is instructed to listen to the USB port. This generates a single sensor whose state rapidly changes every time an ESP-NOW packet arrives.⁵⁵

```
YAML
# sensors.yaml
- platform: serial
  serial_port: /dev/ttyUSB0
  baudrate: 115200
  name: "Raw ESPNOW Stream"
```

7.3 Disaggregating Data via Jinja2 Templating

The data stream pouring into sensor.raw_espnw_stream must be intercepted and split.⁵⁵ Home Assistant relies on Jinja2, a powerful templating engine, to execute this logic. Because the serial data arrives as a concatenated string (e.g., ESPNOW_RCV,1,1,3.92), the split(',') method is invoked to convert the string into an array indexable by position.⁵⁵ Furthermore, because multiple edge nodes will be broadcasting to the same gateway, the

template must conditionally check the DEVICE_ID before updating an entity's state. If a packet arrives for Device 2, the entity representing Device 1 must retain its current state rather than throwing an error or adopting the wrong data.⁵⁸ This is handled using trigger-based template sensors that only evaluate when the base serial sensor undergoes a state change.⁵⁹ The following configuration within templates.yaml demonstrates how to cleanly extract the battery voltage and motion state for an edge node designated as Device ID 1 (e.g., the Balcony Sensor).

YAML

```
# templates.yaml
- trigger:
  - platform: state
    entity_id: sensor.raw_espnw_stream
  sensor:
    - name: "Balcony Sensor Battery"
      unit_of_measurement: "V"
      device_class: battery
      state_class: measurement
      state: >
        {% set payload = states('sensor.raw_espnw_stream').split(',') %}
        {% if payload | length == 4 and payload == 'ESPNOW_RCV' and payload == '1' %}
          {{ payload | float(default=0) }}
        {% else %}
          {{ this.state }}
        {% endif %}

- trigger:
  - platform: state
    entity_id: sensor.raw_espnw_stream
  binary_sensor:
    - name: "Balcony Motion Detected"
      device_class: motion
      state: >
        {% set payload = states('sensor.raw_espnw_stream').split(',') %}
        {% if payload | length == 4 and payload == 'ESPNOW_RCV' and payload == '1' %}
          {% if payload == '1' %}
            true
          {% else %}
            false
```



```

    {% endif %}
    {% else %}
    {{ this.state }}
    {% endif %}
    # Optional: Automatically clear the motion state after 30 seconds
    # auto_off: "00:00:30"

```

This trigger-based paradigm guarantees that the internal state machine of Home Assistant is updated instantaneously without polling delays, providing the millisecond responsiveness required to turn on a light before an individual fully enters a room.⁵⁹

8. Supply Chain Logistics and Hardware Procurement

The theoretical design of an embedded system must eventually confront the realities of supply chain availability. Sourcing authentic, high-quality integrated circuits is a critical factor, particularly concerning the AC-DC conversion and solid-state relay modules where catastrophic failure entails severe physical risk.

For developers implementing this architecture in Bangladesh, procuring parts locally circumvents extensive international shipping delays and customs friction. The domestic market has matured significantly, providing robust access to ESP32 variants and associated passive components.

The ESP32-C3 SuperMini, crucial for the edge nodes, is widely distributed. Vendors such as Electroxbd (situated in Rajshahi), Leetechbd, and BDTronics maintain deep inventories of this specific RISC-V module, allowing for rapid scaling of sensor networks.¹ Procurement of the ESP32-WROOM-32U variant, which includes the necessary U.FL connector for the central gateway's external antenna, is equally accessible through these channels, as are the external 2.4GHz omnidirectional antennas and corresponding pigtail cables.⁶²

Components interfacing with the 220V mains supply require more stringent sourcing. The Hi-Link HLK-PM01 and the Omron G3MB-202P are frequently counterfeited on broad consumer marketplaces. Authentic modules, alongside the mandatory safety components—specifically the 10D561K varistors, 0.5A slow-blow ceramic fuses, and 10-15mH common mode chokes—must be sourced from specialized electronic suppliers rather than general e-commerce platforms to guarantee their thermal and electrical ratings are genuine.³⁴

9. Synthesized Conclusion

The deployment of a custom, highly autonomous smart home network requires an intersection of mechanical, electrical, and software engineering. Bypassing the inherent latencies and energy demands of traditional 802.11 Wi-Fi by leveraging ESP-NOW allows battery-powered edge nodes utilizing the ESP32-C3 architecture to operate invisibly for years. By bridging this RF network directly into a centralized Home Assistant server via a UART serial connection, the entire ecosystem achieves an unparalleled degree of reliability, rendering it entirely immune to router instability, IP lease expiration, and cloud-provider outages.

Simultaneously, the careful application of solid-state relays driven by dedicated, highly protected AC-DC converter modules eliminates the "Spiderman" effect of tangled external

power adapters, replacing them with a sleek, invisible infrastructure hidden entirely within the architecture of the building. Through disciplined hardware debouncing, meticulous C++ memory management, and advanced YAML templating, this architectural blueprint elevates a hobbyist concept into a professional, enterprise-grade deployment capable of sophisticated, instantaneous environmental automation.

Works cited

1. ESP32 C3 super mini - leetechbd, accessed April 30, 2026, <https://leetechbd.com/product/esp32-c3-super-mini/>
2. Getting Started with Seeed Studio XIAO ESP32C3 - Wiki SeeedStudio, accessed April 30, 2026, https://wiki.seeedstudio.com/XIAO_ESP32C3_Getting_Started/
3. ESP32 Motion Sensor Tutorial with ESP32-C3 (TinyML Upgrade) - Fly-Wing - Flywing Tech, accessed April 30, 2026, <https://www.flywing-tech.com/blog/esp32-motion-sensor-tutorial-with-esp32-c3-tinyml-upgrade/>
4. PIR Sensor Modules - Operation, Pinouts, Specs, and Capabilities - Data Capture Control, accessed April 30, 2026, <https://datacapturecontrol.com/articles/io-components/sensors/motion/pir>
5. ESP32 with PIR Motion Sensor using Interrupts and Timers - Random Nerd Tutorials, accessed April 30, 2026, <https://randomnerdtutorials.com/esp32-pir-motion-sensor-interrupts-timers/>
6. Building a Year-Long Battery-Powered WiFi Temperature Sensor | by Lee Harding - Medium, accessed April 30, 2026, <https://medium.com/@lee.harding/building-a-year-long-battery-powered-wifi-temperature-sensor-8b1d1f42542a>
7. Using solar cell & charge module to power esp32 but can't charge, accessed April 30, 2026, <https://electronics.stackexchange.com/questions/732190/using-solar-cell-charge-module-to-power-esp32-but-cant-charge>
8. Power ESP32/ESP8266 with Solar Panels and Battery - Random Nerd Tutorials, accessed April 30, 2026, <https://randomnerdtutorials.com/power-esp32-esp8266-solar-panels-battery-level-monitoring/>
9. GitHub - JackGruber/esp32_ttn_environmental_sensor: Battery/solar-powered ESP32 TTN (LoraWAN) sensor for measuring temperature, humidity, air pressure and fine dust., accessed April 30, 2026, https://github.com/JackGruber/esp32_ttn_environmental_sensor
10. ESP32-C3 Supermini power consumption vs red LED always on - Reddit, accessed April 30, 2026, https://www.reddit.com/r/esp32/comments/1p6nzio/esp32c3_supermini_power_consumption_vs_red_led/
11. Optimising Deep Sleep on ESP32-S3 Super Mini... help needed please - Reddit, accessed April 30, 2026, https://www.reddit.com/r/esp32/comments/1od2bpc/optimising_deep_sleep_on_

[esp32s3_super_mini_help/](#)

12. Modify the ESP32-C3 Super Mini to reduce battery power consumption - YouTube, accessed April 30, 2026,
<https://www.youtube.com/shorts/SdW2X4hqAKU>
13. Controlling ESP32-C3 Super Mini Onboard LEDs - DEV Community, accessed April 30, 2026,
<https://dev.to/gapple/controlling-esp32-c3-super-mini-onboard-leds-2mmn>
14. DIY Solar Battery Charger for 18650 Li-Ion Batteries - Energia Zero, accessed April 30, 2026,
https://energiazero.org/cartelle/arduino/arduino_applicazioni/esp32/electronicshub.org-diy%20solar%20battery%20charger%20for%2018650%20li-ion%20batteries.pdf
15. Solar Power for Arduino/ESP32 : 7 Steps (with Pictures) - Instructables, accessed April 30, 2026, <https://www.instructables.com/Solar-Power-for-ArduinoESP32/>
16. Battery question: ESP32 + TP4056 with a single power source? - Reddit, accessed April 30, 2026,
https://www.reddit.com/r/esp32/comments/1c12ptr/battery_question_esp32_tp4056_with_a_single_power/
17. TP4056 with solar panels : r/esp8266 - Reddit, accessed April 30, 2026,
https://www.reddit.com/r/esp8266/comments/zks2jt/tp4056_with_solar_panels/
18. TP4056 lithium charger module - modification to add power sharing - Arduino Forum, accessed April 30, 2026,
<https://forum.arduino.cc/t/tp4056-lithium-charger-module-modification-to-add-power-sharing/1008473>
19. Design my own TP4056 Li-Ion Charger and IoT Power Supply Subsystem - E-Tinkers, accessed April 30, 2026,
<https://www.e-tinkers.com/2025/08/design-my-own-tp4056-li-ion-charger-and-iot-power-supply-subsystem/>
20. Garden133 Hardware: Solar-Powered, ESP32-Based Soil Moisture Sensor System Design, accessed April 30, 2026,
<https://selectiveappeal.org/posts/garden133-hardware/>
21. PIR Sensors Guide with Arduino Programming for motion detection - Electronic Wings, accessed April 30, 2026,
<https://www.electronicwings.com/sensors-modules/pir-sensor>
22. PIR Motion Sensor: How It Works, Wiring & Applications - Bettlink, accessed April 30, 2026,
<https://www.bettlink.com/blog/pir-motion-sensor-how-it-works-wiring-applications>
23. ESP32-C3 Super Mini Pinout Reference - Last Minute Engineers, accessed April 30, 2026,
<https://lastminuteengineers.com/esp32-c3-super-mini-pinout-reference/>
24. Sleep Modes - ESP32-C3 - — ESP-IDF Programming Guide v6.0.1 documentation, accessed April 30, 2026,
https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-reference/system/sleep_modes.html

25. ESP-NOW - ESP32 - — ESP-IDF Programming Guide v6.0.1 documentation - Espressif Systems, accessed April 30, 2026, https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html
26. eccnrl/ESPNow2Mqtt: library for ESP32 to bridge ESPNow and MQTT - GitHub, accessed April 30, 2026, <https://github.com/eccnrl/ESPNow2Mqtt>
27. Unable to wake the XIAO ESP32C3 up from deep sleep with a PIR sensor, accessed April 30, 2026, <https://forum.seeedstudio.com/t/unable-to-wake-the-xiao-esp32c3-up-from-deep-sleep-with-a-pir-sensor/274099>
28. ESP32 Deep Sleep with Arduino IDE and Wake Up Sources - Random Nerd Tutorials, accessed April 30, 2026, <https://randomnerdtutorials.com/esp32-deep-sleep-arduino-ide-wake-up-sources/>
29. ESP32 Supermini deep-sleep consuming 400uA, accessed April 30, 2026, <https://esp32.com/viewtopic.php?t=44444>
30. ESP-NOW - ESP32-C3 - — ESP-IDF Programming Guide v6.0 documentation, accessed April 30, 2026, https://docs.espressif.com/projects/esp-idf/en/stable/esp32c3/api-reference/network/esp_now.html
31. 3W Ultra-compact Power Module HLK-PM01 230V AC to 5V/3W DC - LaskaKit, accessed April 30, 2026, https://www.laskakit.cz/user/related_files/hlk-pm01-hi-link.pdf
32. How to Use HLK-PM01: Examples, Pinouts, and Specs - Cirkuit Designer Docs, accessed April 30, 2026, <https://docs.cirkuitdesigner.com/component/6077d39f-f28b-45a8-9b40-0419a2274029/hlk-pm01>
33. Design review request : First AC design - Is this dangerous? - Electronics Stack Exchange, accessed April 30, 2026, <https://electronics.stackexchange.com/questions/700543/design-review-request-first-ac-design-is-this-dangerous>
34. Hi-link HLK-PM01 Common Mode Choke JLCPCB and PCBWay. Is it required? - Reddit, accessed April 30, 2026, https://www.reddit.com/r/esp32/comments/1apq1ou/hilink_hlkpm01_common_mode_choke_jlcpcb_and/
35. Using HLK-PM01 safely - General Electronics - Arduino Forum, accessed April 30, 2026, <https://forum.arduino.cc/t/using-hlk-pm01-safely/454425>
36. Solid State Relay (SSR) Primer - Codrey Electronics, accessed April 30, 2026, <https://www.codrey.com/electronic-circuits/solid-state-relay-ssr-primer/>
37. G3MB DATASHEET 8/99 - RS Online, accessed April 30, 2026, <https://docs.rs-online.com/db5e/0900766b8002f991.pdf>
38. G3MB-202P SSR : PinOut, Datasheet & Its Working - WatElectronics.com -, accessed April 30, 2026, <https://www.watelectronics.com/g3mb-202p-solid-state-relay/>
39. G3MB-202P SSR: The Real-World Performance of This Solid-State Relay in DIY and

- Industrial Applications - AliExpress, accessed April 30, 2026,
<https://www.aliexpress.com/s/wiki-ssr/article/g3mb-202p-ssr>
40. ESP32-Based Automatic Bell System with Solid-State Relay (SSR) and RTC – Seeking Feedback & Improvements - Arduino Forum, accessed April 30, 2026,
<https://forum.arduino.cc/t/esp32-based-automatic-bell-system-with-solid-state-relay-ssr-and-rtc-seeking-feedback-improvements/1366790>
 41. SSR relay power switch guidance needed - Arduino Forum, accessed April 30, 2026,
<https://forum.arduino.cc/t/ssr-relay-power-switch-guidance-needed/940285>
 42. Control AC devices with Solid State Relay - Makerguides.com, accessed April 30, 2026, <https://www.makerguides.com/control-ac-devices-with-solid-state-relay/>
 43. Correct diagram to wire G3MB-202P for High trigger using arduino? - Reddit, accessed April 30, 2026,
https://www.reddit.com/r/AskElectronics/comments/1hdbcu9/correct_diagram_to_wire_g3mb202p_for_high_trigger/
 44. ESP-NOW: WiFi With 3x More Range - Instructables, accessed April 30, 2026,
<https://www.instructables.com/ESP-NOW-WiFi-With-3x-More-Range/>
 45. ESP-NOW Two-Way Communication Between ESP32 Boards - Random Nerd Tutorials, accessed April 30, 2026,
<https://randomnerdtutorials.com/esp-now-two-way-communication-esp32/>
 46. Espnow Based Sensors With MQTT/ Home Assistant : 6 Steps - Instructables, accessed April 30, 2026,
<https://www.instructables.com/Espnow-Based-Sensors-With-MQTT-Home-Assistant/>
 47. Getting Started with ESP-NOW (ESP32 with Arduino IDE) - Random Nerd Tutorials, accessed April 30, 2026,
<https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>
 48. ESPNow to Homeassistant/MQTT Gateway : 4 Steps - Instructables, accessed April 30, 2026,
<https://www.instructables.com/ESPNow-to-HomeassistantMQTT-Gateway/>
 49. Channel-hopping on the ESP32-C3 with ESP-Now - DEV Community, accessed April 30, 2026,
<https://dev.to/gtanyware/channel-hopping-on-the-esp32-c3-with-esp-now-2led>
 50. Super Simple ESPNow to Wi-Fi / MQTT Gateway : 7 Steps - Instructables, accessed April 30, 2026,
<https://www.instructables.com/Super-Simple-ESPNow-to-Wi-Fi-MQTT-Gateway/>
 51. WiFi Now component - ESPHome - Home Assistant Community, accessed April 30, 2026, <https://community.home-assistant.io/t/wifi-now-component/449556>
 52. MQTT Gateway integration for ESPHome, which allows for bidirectional bridging between MQTT and an ESPNow network, including automatic entity discovery - Reddit, accessed April 30, 2026,
https://www.reddit.com/r/Esphome/comments/1s61vjr/i_made_an_espnw_mqtt_gateway_integration_for/
 53. ESP32-CAM Connect External Antenna (Extend Wi-Fi Coverage) - Random Nerd Tutorials, accessed April 30, 2026,

- <https://randomnerdtutorials.com/esp32-cam-connect-external-antenna/>
54. Improving WiFi reception with an ESP32-WROOM-32U - Alberto Ruibal, accessed April 30, 2026,
<https://www.alonsoruibal.com/improving-wifi-reception-with-an-esp32-wroom-32u/>
 55. Serial - Home Assistant, accessed April 30, 2026,
<https://www.home-assistant.io/integrations/serial/>
 56. Splitting up the configuration - Home Assistant, accessed April 30, 2026,
https://www.home-assistant.io/docs/configuration/splitting_configuration/
 57. How to split data from serial sensor to sensor with attributes? - Home Assistant Community, accessed April 30, 2026,
<https://community.home-assistant.io/t/how-to-split-data-from-serial-sensor-to-sensor-with-attributes/219740>
 58. Serial Sensor Topic with multiple sensors - Configuration - Home Assistant Community, accessed April 30, 2026,
<https://community.home-assistant.io/t/serial-sensor-topic-with-multiple-sensors/941603>
 59. Template - Home Assistant, accessed April 30, 2026,
<https://www.home-assistant.io/integrations/template/>
 60. ESP32 C3 Super Mini Development Board BT WiFi - ElectroXBD, accessed April 30, 2026,
<https://electroxbd.com/product/esp32-c3-super-mini-dev-board-electroxbd/>
 61. Microcontrollers/ MCU - Active Components - BDTronics, accessed April 30, 2026, <https://www.bdtronics.com/components/active/microcontrollers-mcu.html>
 62. Buy wifi antenna for esp32 Online at Best Price in Bangladesh - Daraz.com.bd, accessed April 30, 2026, <https://www.daraz.com.bd/tag/wifi-antenna-for-esp32/>
 63. G3MB-202P 5VDC Solid State Relay - Dhaka - Daraz.com, accessed April 30, 2026,
<https://www.daraz.com.bd/products/g3mb-202p-5vdc-solid-state-relay-i324845471.html>